

CAPSTONE ASSIGNMENT 1

E-Commerce Web Application Automation

Using Selenium WebDriver with Python

Application Under Test: AutomationExercise

Student Name	Samadrito Samanta
Enrollment / Roll No.	12023002001297
Course / Branch	CSE
Section / Batch	A
Faculty / Trainer	Sramana Bhattacharya
Submission Date	28/09/2026
Institution	Institute Of Engineering And Management(IEM),Kolkata

Technology Stack: Selenium WebDriver | Python | Pytest | pytest-html

Automation Pattern: Page Object Model (POM)

1. Executive Summary

This project presents a maintainable UI automation framework for a representative e-commerce workflow. Selenium WebDriver drives a Chrome browser while Python, Pytest and the Page Object Model organize the test implementation. The automated journey signs in to a test account, searches for a product, applies a requested quantity, adds the product to the cart, checks the resulting cart data, captures evidence screenshots and produces an HTML test report.

A reusable cart-state cleanup step is included before the purchase flow. This prevents a previous execution of the same test account from changing the expected quantity in the current run. Test inputs are externalized in JSON so the scenario can be updated without changing the test logic.

2. Scenario and Scope

The business scenario models a customer purchasing a product from a public training e-commerce site. The automation focuses on the customer-facing UI rather than payment processing or a real transaction.

- Open the AutomationExercise site in Chrome.
- Authenticate using credentials stored outside the test code.
- Navigate to the product catalogue and search for Blue Top.
- Open the selected product and set the requested quantity to 2.
- Add the product to the cart and handle the application popup / native alert when present.
- Verify product name, unit price, quantity and calculated line total.
- Capture screenshots at important checkpoints and preserve failure evidence.
- Generate an HTML execution report through pytest-html.

3. Requirement-to-Implementation Matrix

Requirement	Implementation	Project Location
Launch browser	Chrome WebDriver is created by a Pytest fixture.	conftest.py
Login	Page object enters email/password and submits the login form.	pages/login_page.py
Search product	Search field and result verification are encapsulated in a page object.	pages/products_page.py
Add to cart	Product page performs quantity entry and Add to Cart.	pages/product_details_page.py
Update quantity	Quantity is read from external JSON test data.	testdata/test_data.json
Verify cart	Name, price, quantity and line total are asserted.	pages/cart_page.py
Avoid stale cart data	Existing product is removed before the purchase flow.	tests/test_ecommerce_flow.py
Capture screenshots	Checkpoint and failure screenshots are stored with utility helpers.	utils/screenshot.py
Read test data	JSON is loaded once through a session-level fixture.	utils/data_reader.py / conftest.py
Handle popup / alerts	Explicit waits and optional native-alert handling are used.	BasePage / ProductDetailsPage
Generate report	Pytest-html generates a standalone HTML execution report.	pytest.ini

4. Framework Architecture

The framework separates test intent from browser mechanics. The test class coordinates the business journey, page objects own locators and page-specific actions, BasePage provides reusable Selenium operations, utilities manage common support functions, and the Pytest configuration controls setup, teardown and reporting.

Test Layer → business scenario and assertions
Page Objects → Home, Login, Products, Product Details and Cart
Base Page → waits, click, typing, scrolling and alert handling
Utilities → JSON reader and screenshot helper
Pytest Fixtures → WebDriver lifecycle, configuration and failure evidence
Browser → Chrome controlled by Selenium WebDriver

5. Project Components

Component	Purpose
conftest.py	Creates the driver fixture, loads JSON configuration, applies Chrome options, manages waits, captures failure screenshots and quits the browser.
config/settings.py	Stores project-level paths for test data, screenshots and reports.
utils/data_reader.py	Reads the external JSON test data used by the suite.
utils/screenshot.py	Creates timestamped screenshots for test evidence.
pages/base_page.py	Provides reusable wait, click, typing, visibility, scrolling and optional alert operations.
pages/home_page.py	Handles high-level navigation and verifies the logged-in state.
pages/login_page.py	Encapsulates login form interaction.
pages/products_page.py	Searches products and opens the selected product.
pages/product_details_page.py	Sets quantity, adds the item to the cart and handles the add-to-cart popup.
pages/cart_page.py	Locates cart rows, reads values and validates price, quantity and total.
tests/test_ecommerce_flow.py	Coordinates the complete end-to-end scenario and assertions.
pytest.ini	Configures Pytest discovery and pytest-html reporting.
requirements.txt	Lists the Python packages needed to execute the framework.
testdata/test_data.json	Stores application settings, credentials and product expectations.

6. Test Data and Validation Rules

The scenario uses external JSON data. Keeping values outside the test code improves maintainability and makes the same test flow reusable with different inputs.

```
{
  "application": {"base_url": "https://automationexercise.com/", "browser": "chrome",
    "headless": false, "timeout": 30},
  "credentials": {"email": "<blank-or-valid-test-account>", "password": "<blank-or-valid-test-account-password>"},
  "product": {"search_term": "Blue Top", "expected_name": "Blue Top",
    "expected_price": "Rs. 500", "quantity": 2}
}
```

The cart assertion also checks the arithmetic relationship between the unit price and the displayed line total. With the configured product data, the expected calculation is Rs. 500 × 2 = Rs. 1000.

7. Synchronization and Reliability Measures

- Explicit waits are used when the test needs a page element to become visible, clickable or present.
- A small implicit wait provides limited resilience for ordinary element lookups without replacing explicit synchronization.
- Chrome uses the eager page-load strategy so navigation can continue after the DOM becomes usable rather than waiting for every resource.
- Optional native alerts are accepted only when present, avoiding failures when no alert is generated.
- The add-to-cart modal is handled explicitly before navigation continues.
- The fixture captures a screenshot when the main test fails.
- Cart cleanup removes a pre-existing copy of the target product so repeated executions are independent of previous cart state.

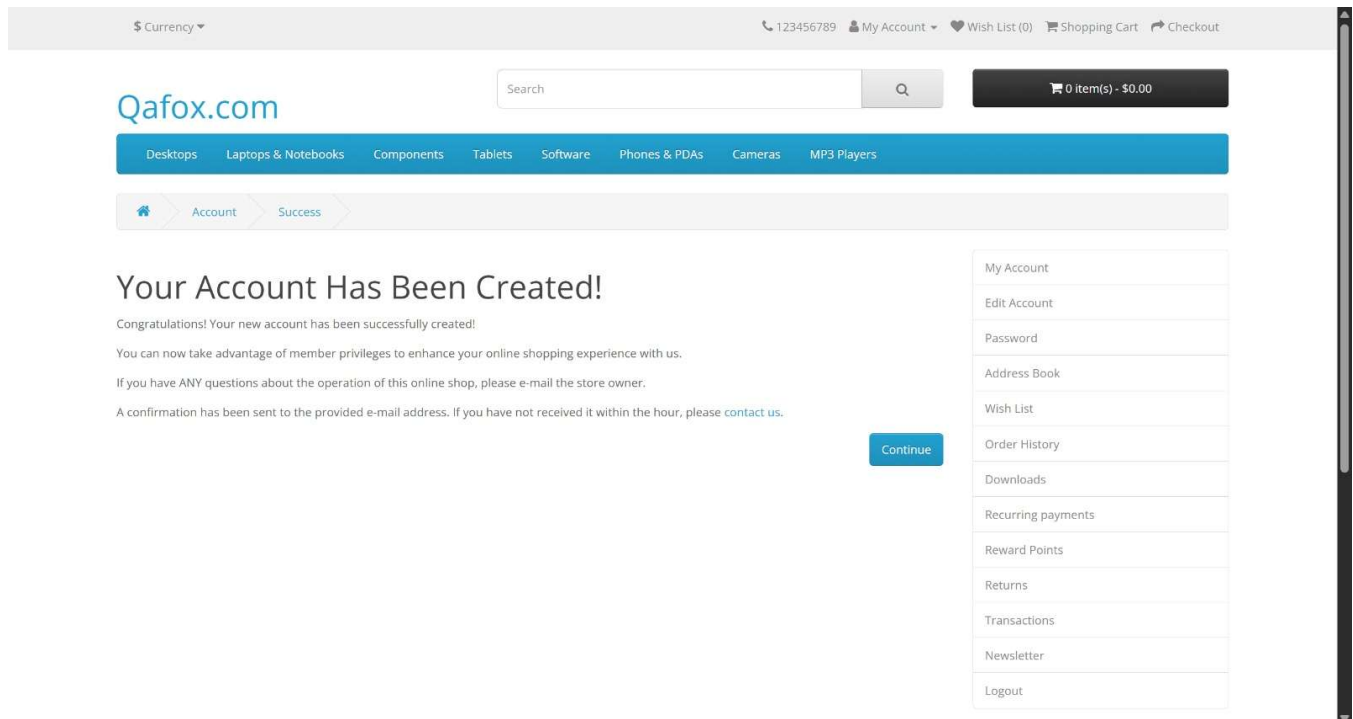
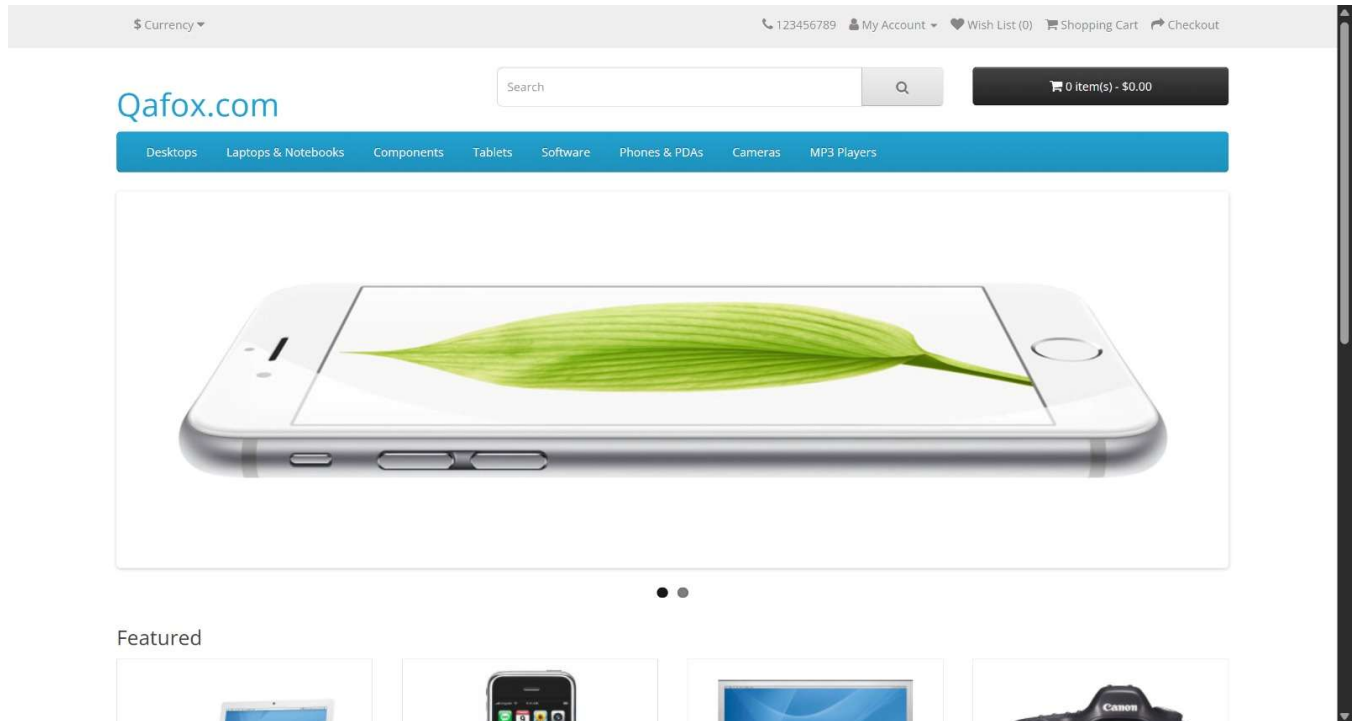
8. End-to-End Execution Flow

Step	Stage	Description
1	Start	Pytest launches the Chrome WebDriver.
2	Open application	AutomationExercise is opened and the title is checked.
3	Login	Credentials from JSON are entered and the logged-in state is verified.
4	Normalize cart	The cart is opened and an existing Blue Top is removed when present.
5	Search	The Products page searches for Blue Top and confirms a matching result.
6	Select	The test opens the chosen product details page.
7	Set quantity	Quantity is changed to 2 using the JSON value.
8	Add to cart	The product is added and the application popup / alert is handled.
9	Verify	Cart values are read and checked against name, price, quantity and total.
10	Report	Screenshots are saved and pytest-html generates the execution report.

9. Expected Outcomes

- AutomationExercise opens successfully and the page title is validated.

- The configured test account reaches the logged-in state.
- Blue Top is found through the product search.
- The selected product is added with quantity 2.
- The cart shows Blue Top at Rs. 500 per unit, quantity 2 and a line total of Rs. 1000.
- Checkpoint screenshots are stored under screenshots/.
- The execution summary is produced as reports/execution_report.html.



Currency

123456789My AccountWish List (0)Shopping CartCheckout

Qafox.com

Search

0 item(s) - \$0.00

DesktopsLaptops & NotebooksComponentsTabletsSoftwarePhones & PDAsCamerasMP3 Players

Account

My Account

Edit your account information

Change your password

Modify your address book entries

Modify your wish list

My Orders

View your order history

Downloads

Your Reward Points

View your return requests

Your Transactions

Recurring payments

My Affiliate Account

Register for an affiliate account

Newsletter

Subscribe / unsubscribe to newsletter

My Account

Edit Account

Password

Address Book

Wish List

Order History

Downloads

Recurring payments

Reward Points

Returns

Transactions

Newsletter

Logout

Currency

123456789My AccountWish List (0)Shopping CartCheckout

Qafox.com

iPhone

0 item(s) - \$0.00

DesktopsLaptops & NotebooksComponentsTabletsSoftwarePhones & PDAsCamerasMP3 Players

Search

Search - iPhone

Search Criteria

iPhone

All Categories

Search in subcategories

Search in product descriptions

Search

Products meeting the search criteria

Product Compare (0)

Sort By: Default

Show: 20



All Categories

☐ Search in subcategories

☐ Search in product descriptions

Search

Products meeting the search criteria

Product Compare (0)

Sort By: Default

Show: 20



iPhone

iPhone is a revolutionary new mobile phone that allows you to make a call by simply tapping a name o..

\$123.20

Ex Tax:\$101.00

ADD TO CART

Showing 1 to 1 of 1 (1 Pages)

\$ Currency
123456789
My Account
Wish List (0)
Shopping Cart
Checkout

Qafox.com

Q

1 Item(s) - \$123.20

Desktops Laptops & Notebooks Components Tablets Software Phones & PDAs Cameras MP3 Players

Shopping Cart

Products marked with *** are not available in the desired quantity or not in stock!

Shopping Cart (10.00kg)

Image	Product Name	Model	Quantity	Unit Price	Total
	iPhone ***	product 11	1	\$123.20	\$123.20

What would you like to do next?

Choose if you have a discount code or reward points you want to use or would like to estimate your delivery cost.

Use Coupon Code

Estimate Shipping & Taxes

Use Gift Certificate

Currency

123456789My AccountWish List (0)Shopping CartCheckout

Qafox.com

Search

3 item(s) - \$369.60

DesktopsLaptops & NotebooksComponentsTabletsSoftwarePhones & PDAsCamerasMP3 Players

Shopping Cart

Success: You have modified your shopping cart!

Products marked with *** are not available in the desired quantity or not in stock!

Shopping Cart (30.00kg)

Image	Product Name	Model	Quantity	Unit Price	Total
	iPhone ***	product 11	3	\$123.20	\$369.60

What would you like to do next?

Choose if you have a discount code or reward points you want to use or would like to estimate your delivery cost.

Use Coupon Code

Estimate Shipping & Taxes

Currency

123456789My AccountWish List (0)Shopping CartCheckout

Qafox.com

Search

3 item(s) - \$369.60

DesktopsLaptops & NotebooksComponentsTabletsSoftwarePhones & PDAsCamerasMP3 Players

Shopping Cart

Products marked with *** are not available in the desired quantity or not in stock!

Shopping Cart (30.00kg)

Image	Product Name	Model	Quantity	Unit Price	Total
	iPhone ***	product 11	3	\$123.20	\$369.60

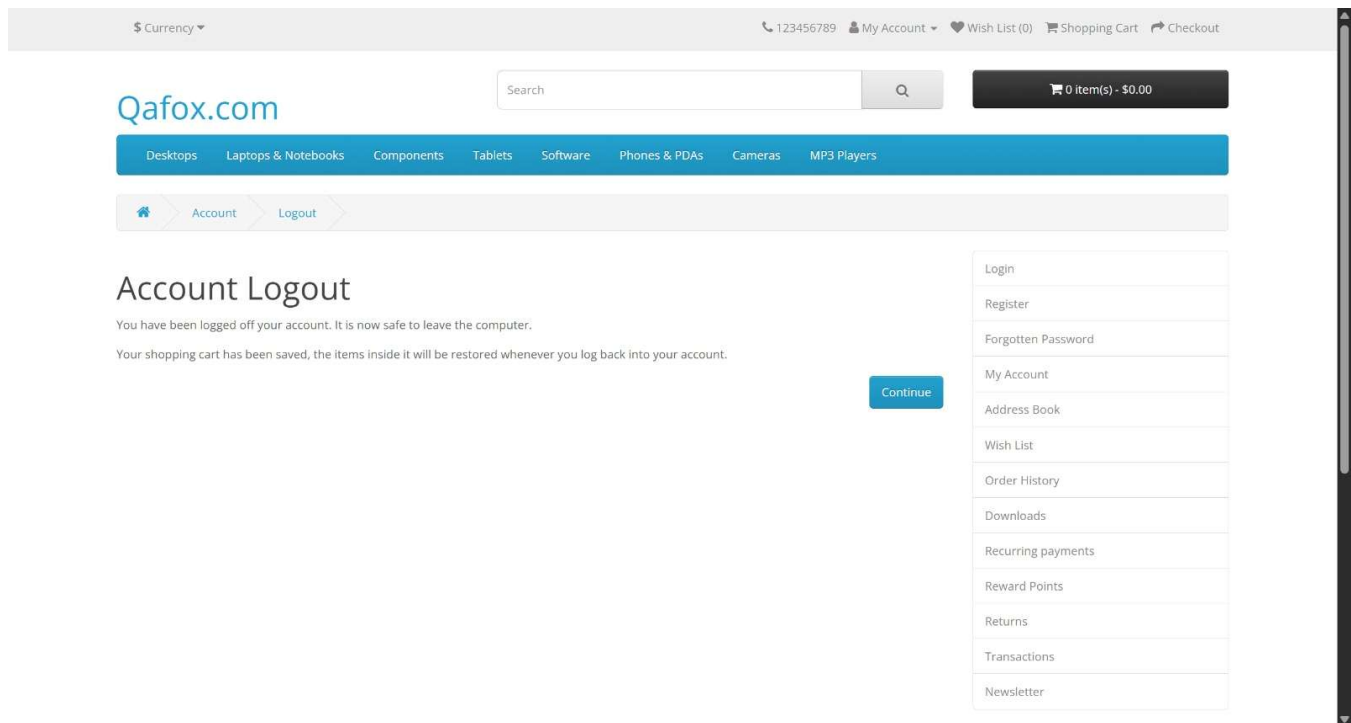
What would you like to do next?

Choose if you have a discount code or reward points you want to use or would like to estimate your delivery cost.

Use Coupon Code

Estimate Shipping & Taxes

Use Gift Certificate



10. Execution Procedure

```
python -m venv .venv  
  
.venv\Scripts\activate  
  
pip install -r requirements.txt  
  
python -m pytest -v  
  
start reports\execution_report.html
```

The test should be executed with `python -m pytest` rather than calling `pytest.exe` directly on systems where Windows application-control policy blocks the executable launcher.

11. Limitations and Assumptions

- The automation depends on the availability and current DOM structure of the public AutomationExercise site.
- A valid test account is required for the login flow.
- The framework is configured for Chrome in the supplied setup.
- The scenario validates the cart stage and does not perform a real payment or financial transaction.
- Selectors may require maintenance if the application UI changes.

12. Conclusion

The completed framework demonstrates a practical Selenium WebDriver solution for an e-commerce workflow using Python and Pytest. Page Object Model isolates UI-specific code, external JSON keeps test inputs configurable, reusable utilities provide screenshots and supporting actions, and pytest-html produces a reviewable execution artifact. The additional cart-normalization step makes repeated executions more reliable by preventing stale cart data from affecting the expected quantity.

13. References

- AutomationExercise: <https://automationexercise.com/>
- AutomationExercise Test Cases: https://www.automationexercise.com/test_cases
- AutomationExercise Login: <https://automationexercise.com/login>
- AutomationExercise Products: <https://automationexercise.com/products>
- Selenium documentation: <https://www.selenium.dev/documentation/>